INSTITUTO

IES 9-008 Manuel Belgrano

VELOCRAFT

Catálogo y Administración de Cuadros de Bicicleta de Alta Gama

CARRERA

Arquitectura y diseño de interfaces

CURSO

3-2

INTEGRANTES

Jonatán Santis

Ezequiel Puebla

Méndez Lucas

Junio, 2026

Documentación del Proyecto: VeloCraft

👥 Entrevista Número: 01 (Alineación y Definición de Alcance)

- **Fecha:** 4 de junio de 2026
- **Participantes:** Cliente (Usuario) / Equipo de Desarrollo
- **Objetivo:** Definir los alcances funcionales, de diseño y tecnológicos para el catálogo interactivo de cuadros de bicicleta.
- **Acuerdos clave:**
  - La aplicación debe funcionar sin backend (Single Page Application offline).
  - Los datos se persistirán localmente en el navegador para simular el almacenamiento en base de datos.
  - El diseño visual debe transmitir exclusividad técnica (estilo oscuro de alta gama).

📋 Requerimientos Iniciales

1. **Separación Limpia de Archivos:** Arquitectura modular basada en `index.html` para marcado, `style.css` para hojas de estilo y `app.js` para controladores y lógica del modelo.
2. **Semántica HTML5:** Código bien indentado estructurado con cabecera (`<header>`), sección principal (`<main>`), barra lateral collapsible (`<aside>`) y pie de página (`<footer>`).
3. **Responsive Grid:** Adaptabilidad móvil nativa utilizando componentes y grillas flexibles de Bootstrap 5.
4. **Validaciones:** Control de errores contra registros inválidos o vacíos directamente en el frontend.

💡 Necesidades Planteadas

El cliente requiere un sistema dinámico que sirva tanto de escaparate virtual (vistas para compradores) como de consola de control de inventario (vistas de administración). Se requiere que un administrador pueda agregar nuevos productos, alterar especificaciones existentes, y dar de baja ítems obsoletos, con un cálculo automático del valor totalizado de los productos almacenados en stock.

🚲 Modelos Destacados e Imágenes del Proyecto

Renders de alta definición generados e integrados en el catálogo dinámico de la aplicación:

MODELO 1: VELOCRAFT CARBON SLR

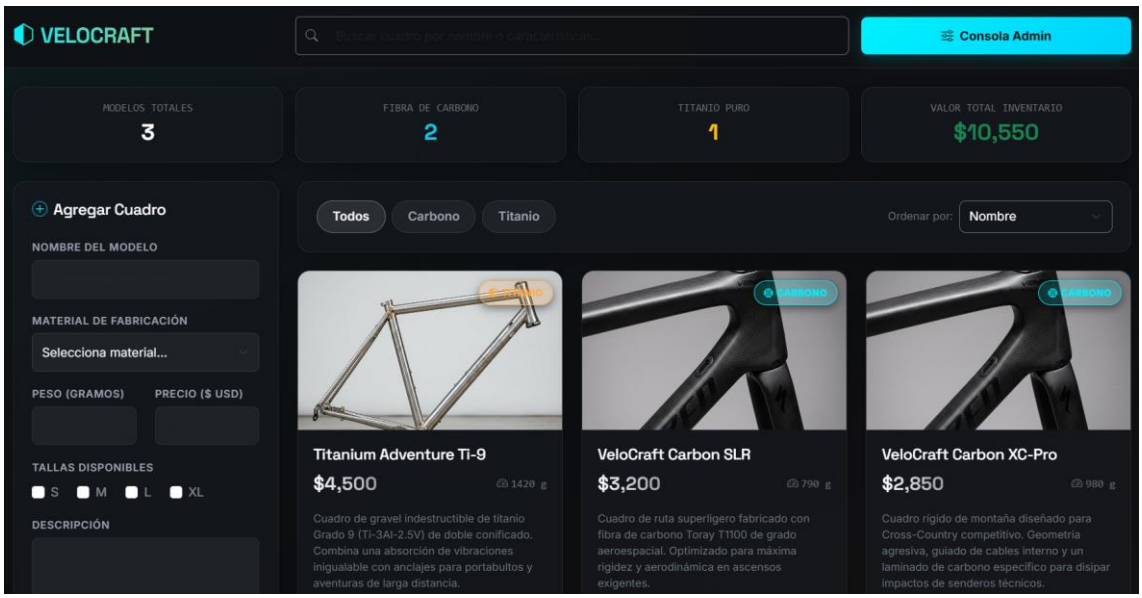


Estructura aerodinámica de carbono Toray

MODELO 2: TITANIUM ADVENTURE TI-9



Titanio Grado 9 de doble conificado para gravel.



Requerimientos Funcionales (RF)

ID	Nombre	Descripción	Prioridad
RF-01	Visualización de Catálogo	Carga y renderizado dinámico de tarjetas de productos desde el almacenamiento.	Alta
RF-02	Filtrado por Material	Botones interactivos para segregar cuadros de carbono y titanio al instante.	Alta
RF-03	Búsqueda en Tiempo Real	Input de texto que filtre coincidencias por nombre o descripción técnica.	**Media**
RF-04	Registro de Cuadros (C)	Formulario con validación para dar de alta nuevos cuadros con nombre, material, peso, precio, tallas y descripción.	Alta
RF-05	Edición de Cuadros (U)	Modal interactivo para modificar los parámetros de los cuadros existentes.	Alta
RF-06	Eliminación de Cuadros (D)	Botón para remover permanentemente cuadros del catálogo con confirmación de seguridad.	Alta
RF-07	Dashboard de Estadísticas	Cálculo de cuadros totales, conteo por material y valor acumulado del inventario.	**Media**
RF-08	Persistencia Local	Sincronización automática de todas las transacciones mediante localStoragede HTML5.	Alta

⚙ Requerimientos NO Funcionales (RNF)

ID	Nombre	Descripción	Métrica / Estándar
RNF-01	Responsividad	La interfaz debe adaptarse fluidamente a dispositivos móviles, tablets y monitores de escritorio.	Bootstrap breakpoints (col-12, col-md-6, col-lg-4)
RNF-02	Rendimiento	Las operaciones de filtrado, render y ordenamiento deben ocurrir instantáneamente sin refrescar la página.	Tiempo de ejecución en JavaScript < 50ms
RNF-03	Estética Premium	Paleta de colores oscuros inspirada en carbono y metal, efectos de desenfoque (glassmorphism) y transiciones animadas.	Uso de degradados HSL y animaciones CSS @keyframes
RNF-04	Usabilidad / Feedback	Feedback visual claro al usuario cuando ocurra una modificación en el sistema.	Alertas dinámicas mediante Bootstrap Toast notifications
RNF-05	Compatibilidad	Ejecución compatible en navegadores modernos (Chrome, Firefox, Safari, Edge) utilizando estándares W3C.	Estructuración semántica válida de HTML5

Historias de Usuario

Historia 1: Exploración del Catálogo

COMO cliente interesado en ciclismo de ruta y gravel, **QUIERO** visualizar la galería de cuadros disponibles y poder ordenarlos por precio o filtrar por material,
PARA encontrar la opción que mejor se ajuste a mi presupuesto y estilo de conducción.

Criterios de Aceptación:
- Al abrir la web, se muestran los cuadros disponibles por defecto.
- Puedo seleccionar "Carbono" o "Titanio" y ver solo esos materiales.
- Puedo seleccionar ordenar de "Menor a Mayor Precio" y el catálogo se reordena al instante.

Historia 2: Control y Registro de Inventario

COMO administrador de la tienda,
QUIERO agregar un nuevo modelo de cuadro completando un formulario de especificaciones técnicas,
PARA mantener el catálogo en línea actualizado con los últimos ingresos.

Criterios de Aceptación:
- El formulario requiere obligatoriamente nombre, precio, peso, descripción y al menos una talla.
- Si intento enviar el formulario vacío o con rangos de peso o precio incorrectos, el sistema marca el error y evita la creación.
- Tras un registro exitoso, se limpia el formulario y aparece una alerta flotante de éxito.

Historia 3: Ajustes de Catálogo (Edición)

COMO administrador de la tienda,
QUIERO editar los datos de un cuadro existente en una ventana emergente sin salir de la página principal,
PARA ajustar precios o corregir errores tipográficos de forma ágil.

Criterios de Aceptación:
- Al hacer clic en editar, un modal muestra los valores actuales precargados.
- Al guardar los cambios, la tarjeta del catálogo se actualiza automáticamente y los datos persisten tras refrescar la web.

Historia 4: Eliminación de Modelos

COMO administrador de la tienda,
QUIERO poder eliminar del sistema aquellos cuadros descontinuados o vendidos,
PARA evitar que los clientes soliciten piezas no disponibles.

Criterios de Aceptación:
- Al presionar eliminar, se solicita una confirmación al administrador.
- Si se confirma, la tarjeta se desvanece de la pantalla y el valor total del inventario se actualiza en tiempo real.

MEJORAS Y DESARROLLOS A FUTURO

1. Almacenamiento permanente y centralizado

Actualmente los datos se guardan en el navegador (localStorage), por lo que solo están disponibles en esa computadora. Se puede mejorar agregando:
- Base de datos: Usar MySQL, SQLite o MongoDB para guardar la información de forma segura y permanente
- Servidor: Implementar Node.js + Express para conectar la aplicación con la base de datos
- Acceso desde cualquier lugar: Al tener servidor, el inventario se podría consultar y actualizar desde cualquier dispositivo con internet

2. Gestión de usuarios y seguridad

Para uso profesional, se puede agregar:
- Inicio de sesión: Distinguir entre administradores (que pueden modificar todo) y usuarios de consulta (que solo ven la información)
- Contraseñas cifradas: Proteger el acceso a los datos
- Registro de actividades: Guardar quién y cuándo modificó cada registro

3. Funcionalidades avanzadas

- Subida real de imágenes: Actualmente usa enlaces de internet; se puede agregar la opción de seleccionar y guardar archivos de imagen desde la computadora
- Control de stock: Agregar un campo de cantidad disponible y alertas cuando se queden sin modelos
- Historial de precios: Ver la evolución del valor de cada cuadro
- Generación de reportes: Crear listados o informes en PDF/Excel para imprimir o compartir- Cálculo automático: Conversión de monedas, cálculo de impuestos o costos de envío

4**. Mejoras técnicas y de experiencia**

- Modo oscuro/claro: Opción para cambiar la apariencia
- Respaldo y restauración: Botón para descargar una copia de seguridad de todos los datos
- Búsqueda más inteligente: Autocompletado, filtros combinados y ordenamiento personalizado

- Pruebas automáticas: Asegurar que todas las funciones sigan andando al agregar cambios nuevos

Planificación del Sprint

Sprint 1 (Duración: 1 Semana)
Meta del Sprint: Implementar la Single Page Application interactiva con CRUD completo para la administración de cuadros de bicicleta y diseño responsive.

Día	Tareas del Backlog	Responsables/Hito
Día 1	Estructuración semántica del index.html, inclusión de librerías CDN externas (Bootstrap 5 y Bootstrap Icons).	Frontend Dev - Estructura Base
Día 2	Desarrollo de estilos personalizados en style.css (Paleta de colores, degradados, glassmorphism, responsive breakpoints).	CSS Specialist - Interfaz Visual
Día 3-4	Implementación del modelo FrameInventoryModel en app.js. Funciones CRUD, guardado en LocalStorage y render dinámico de cartas.	Javascript Dev - CRUD Logic
Día 5	Lógica de filtros interactivos por material, ordenamiento, búsqueda en tiempo real, Toasts de notificación y cálculos del dashboard de estadísticas.	Javascript Dev - Controller

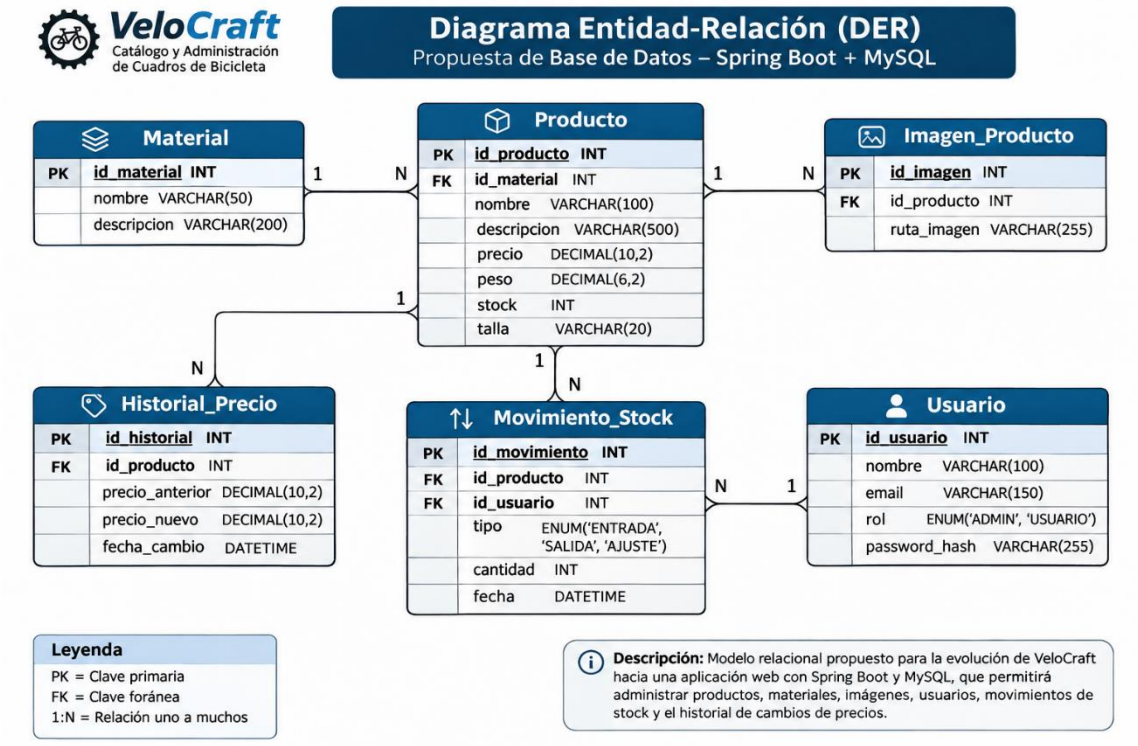
Día 6-7	Verificación de funcionamiento responsivo. Control de errores e input validación. Aseguramiento de consistencia de estados.	QA / Testing - Lanzamiento

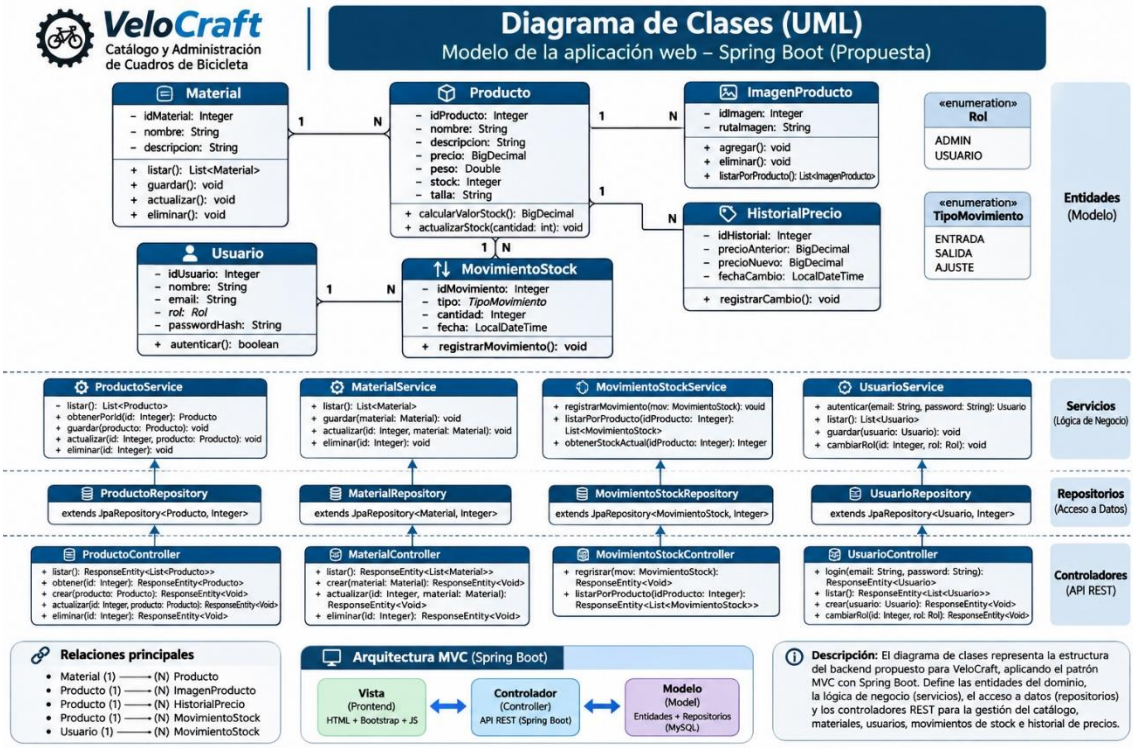
1. Diagrama Entidad-Relación (DER) propuesto

Para VeloCraft propongo estas seis entidades:

Entidad	Función
Producto	Almacena los cuadros de bicicletas, precios, pesos y stock.
Material	Administra materiales como carbono, titanio y aluminio.
ImagenProducto	Permite asociar varias fotografías a un cuadro.
Usuario	Gestiona administradores y usuarios del sistema.
MovimientoStock	Registra entradas, salidas y ajustes de inventario.
HistorialPrecio	Guarda los cambios de precios de cada producto.

Las seis forman parte del modelo relacional futuro, no de una base MySQL que ya esté implementada.




VeloCraft
Catálogo y Administración de Cuadros de Bicicleta
Diagrama de Clases (UML)
Modelo de la aplicación web – Spring Boot (Propuesta)
Material
- idMaterial: Integer
- nombre: String
- descripcion: String
+ listar(): List<Material>
+ guardar(): void
+ actualizar(): void
+ eliminar(): void
Producto
- idProducto: Integer
- nombre: String
- descripcion: String
- precio: BigDecimal
- peso: Double
- stock: Integer
- talla: String
+ calcularValorStock(): BigDecimal
+ actualizarStock(cantidad: int): void
ImagenProducto
- idImagen: Integer
- rutaImagen: String
+ agregar(): void
+ eliminar(): void
+ listarPorProducto(): List<ImagenProducto>
«enumeration» Rol
ADMIN
USUARIO
«enumeration» TipoMovimiento
ENTRADA
SALIDA
AJUSTE
Usuario
- idUsuario: Integer
- nombre: String
- email: String
- rol: Rol
- passwordHash: String
+ autenticar(): boolean
MovimientoStock
- idMovimiento: Integer
- tipo: TipoMovimiento
- cantidad: Integer
- fecha: LocalDateTime
+ registrarMovimiento(): void
HistorialPrecio
- idHistorial: Integer
- precioAnterior: BigDecimal
- precioNuevo: BigDecimal
- fechaCambio: LocalDateTime
+ registrarCambio(): void
1
N
Entidades (Modelo)
ProductoService
- listar(): List<Producto>
+ obtenerPorId(id: Integer): Producto
+ guardar(producto: Producto): void
+ actualizar(id: Integer, producto: Producto): void
+ eliminar(id: Integer): void
MaterialService
+ listar(): List<Material>
+ guardar(material: Material): void
+ actualizar(id: Integer, material: Material): void
+ eliminar(id: Integer): void
MovimientoStockService
+ registrarMovimiento(mov: MovimientoStock): void
+ listarPorProducto(idProducto: Integer): List<MovimientoStock>
+ obtenerStockActual(idProducto: Integer): Integer
UsuarioService
+ autenticar(email: String, password: String): Usuario
+ listar(): List<Usuario>
+ guardar(usuario: Usuario): void
+ cambiarRol(id: Integer, rol: Rol): void
Servicios (Lógica de Negocio)
ProductoRepository
extends JpaRepository<Producto, Integer>
MaterialRepository
extends JpaRepository<Material, Integer>
MovimientoStockRepository
extends JpaRepository<MovimientoStock, Integer>
UsuarioRepository
extends JpaRepository<Usuario, Integer>
Repositorios (Acceso a Datos)
ProductoController
+ listar(): ResponseEntity<List<Producto>>
+ obtener(id: Integer): ResponseEntity<Producto>
+ crear(producto: Producto): ResponseEntity<Void>
+ actualizar(id: Integer, producto: Producto): ResponseEntity<Void>
+ eliminar(id: Integer): ResponseEntity<Void>
MaterialController
+ listar(): ResponseEntity<List<Material>>
+ crear(material: Material): ResponseEntity<Void>
+ actualizar(id: Integer, material: Material): ResponseEntity<Void>
+ eliminar(id: Integer): ResponseEntity<Void>
MovimientoStockController
+ regrisrar(mov: MovimientoStock): ResponseEntity<Void>
+ listarPorProducto(idProducto: Integer): ResponseEntity<List<MovimientoStock>>
UsuarioController
+ login(email: String, password: String): ResponseEntity<Usuario>
+ listar(): ResponseEntity<List<Usuario>>
+ crear(usuario: Usuario): ResponseEntity<Void>
+ cambiarRol(id: Integer, rol: Rol): ResponseEntity<Void>
Controladores (API REST)
Relaciones principales
Material (1) ⟶ (N) Producto
Producto (1) ⟶ (N) ImagenProducto
Producto (1) ⟶ (N) HistorialPrecio
Producto (1) ⟶ (N) MovimientoStock
Usuario (1) ⟶ (N) MovimientoStock
Arquitectura MVC (Spring Boot)
Vista (Frontend) HTML + Bootstrap + JS
Controlador (Controller) API REST (Spring Boot)
Modelo (Model) Entidades + Repositorios (MySQL)
Descripción: El diagrama de clases representa la estructura del backend propuesto para VeloCraft, aplicando el patrón MVC con Spring Boot. Define las entidades del dominio, la lógica de negocio (servicios), el acceso a datos (repositorios) y los controladores REST para la gestión del catálogo, materiales, usuarios, movimientos de stock e historial de precios.